
Vibe Code Academy

Lesson 7: GitHub Branching and Collaboration

Key Terms



1. Your own lane

Branch: your own parallel copy of the code, inside the same repo. You can change anything without touching anyone else's work.

Main: the default/"real" branch that Vercel puts live.

2. Moving Between Lanes

Switch: jump from one branch to another. Your files instantly change to match the branch you're on (older tutorials call this checkout — same thing.)

3. Joining lanes back together

Merge conflict: two branches changed the same line, and Git needs a human to pick which version wins. Totally normal, you didn't break anything.

Branch vs Fork – don't mix these up

• • : : : . . .



Fork

Where it lives:

On your GitHub account, completely separate from the original.

You've done this:

Last week's homework when you forked an open source project.

Use When:

Contributing to a project that you are not a part of.



Branch

Where it lives:

Inside the repo that you and your whole team share.

One repo, many branches:

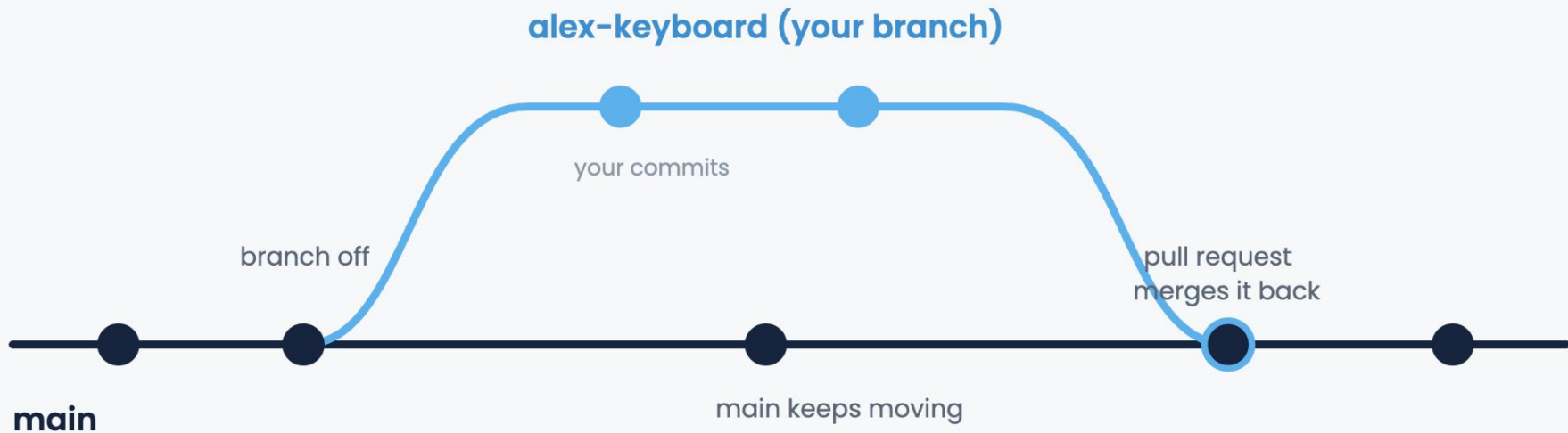
Each teammate works in their own lane, contributing to the same final project, at the same time.

Use When:

Working with other people.

Today you will share one repo with a team, so you will use branches, not forks.

How Branching Works



Main never sees your work until the merge. Your branch splits off, collects commits, and returns through a pull request.

Step 1: Make Your Own Branch



1. Create It On Github

On your team's repo, click the **branch dropdown** (it says **main**), type a name like alex-keyboard, then click **Create branch**: alex-keyboard from main.

2. Make Your Changes

Write the code needed for your branch feature to work.

*The entire process can actually be done just using Claude Code (look to the right)!

3. Check It Worked

After you push your branch, a yellow banner appears on the repo page: **Compare & pull request**. That's your branch arriving on GitHub.

Claude Code Track – Paste These Prompts

1. Branch First

"Create a branch called <your branch name> and switch to it."

2. Build

Use Claude Code as normal, tell it what changes you want it to make or what feature you want the program to have and it will create the code.

3. Push

"Commit all the changes in this session to the branch and push it to GitHub."

*After you get Claude to push it to GitHub, go to step three on the left.

Step 2: Pull Request and Merging

• • • • •

1. Open PR

After your push, a yellow **Compare & pull request** banner appears on the repo page. Click it, write one line about your change, then click **Create pull request**.

2. Teammate Reviews

A teammate opens the **Files changed** tab: green lines were added, red lines removed. They comment, suggest edits, or say it looks good.

3. Merge It

Teammate will click **Merge pull request** → **Confirm merge**. Your branch joins main — and Vercel redeploys the live site automatically.



It's that easy. But remember, step two can only be done by you, not Claude. However, you can ask Claude to help you review the changes that your teammates made.

Merge Conflicts – You Will Hit One

• • : : : . . .

What It Looks Like

```
<<<<<<<< HEAD <h1>Wordle by Team 3</h1> =====  
<h1>Team 3 Wordle</h1> >>>>>>> alex-keyboard
```

Git shows you both versions: what's on main (top) and what's on your branch (bottom)

Why It Happens

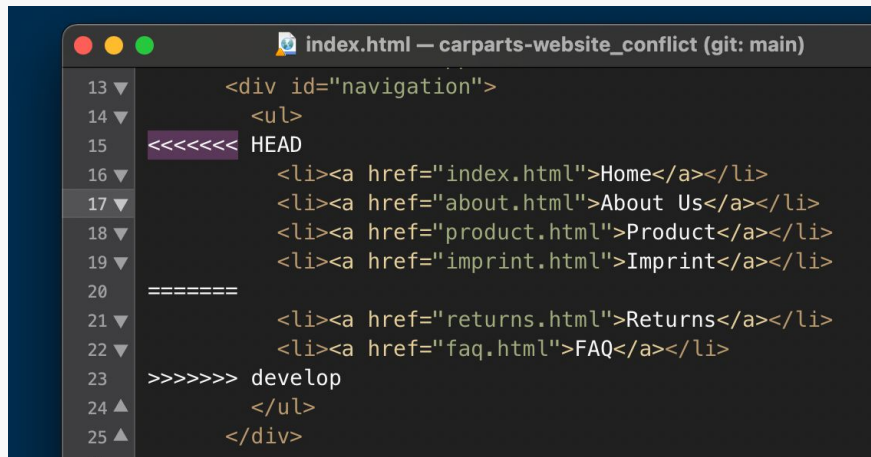
This tends to happen when two branches change the **same line**. Instead of assuming which is correct, Git stops and asks you.

How To Fix It

Keep the version you want, or combine both versions of code. Delete the <<< === >>> marker lines, then commit again.

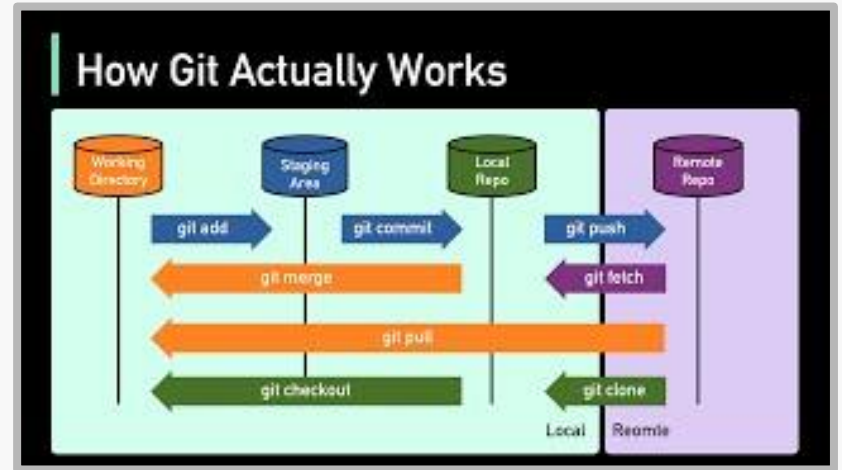
The Escape Hatch

Paste the entire block of code into Claude Code then say, "I have a merge conflict in <name of the file>. Please help me resolve it and keep all features working."



```
13 <div id="navigation">  
14 <ul>  
15 <<<<<<<< HEAD  
16 <li><a href="index.html">Home</a></li>  
17 <li><a href="about.html">About Us</a></li>  
18 <li><a href="product.html">Product</a></li>  
19 <li><a href="imprint.html">Imprint</a></li>  
20 =====  
21 <li><a href="returns.html">Returns</a></li>  
22 <li><a href="faq.html">FAQ</a></li>  
23 >>>>> develop  
24 </ul>  
25 </div>
```

Watch These Two Videos



Task: Recreate Wordle



1. Teams of Three

Breakout rooms, three per group. **Repo owner goes first:** add your teammates under **Settings** → **Collaborators**

2. Build a Good Clone

You are free to use any tools you like - Claude Design, Claude Code, Lovable. Split the work: one person works on the board, one on keyboard, and one on game logic.

3. Ship It As You Go

Connect the repo to **Vercel** at the **start** of the process, not end. This way every merged PR updates the live site.

Must Have Requirements

1. Everyone Works On Their Own Branch

Nobody should be pushing their work directly to main. The objective of this task is to learn how to collaborate with others while coding.

2. 2+ Pushes Per Person

Everyone should push their branch to GitHub at least twice. By doing this, you will get a better understanding of how the mechanic works.

3. 2+ Merge PR Per Team

This will teach you how to merge a PR and how to overcome merge conflicts should it come up.

Don't hesitate to reach out to a tutor or ask AI if you have a question or are getting stuck!

Next Lesson: Authentication and APIs

Homework: Finish your Wordle copy as a team. After it is done, the repo owner will DM the team's Vercel + Repo link to Yibo, Rishabh, or Nirvan on Slack. We will be looking at repo history (branches and merged PR), not just the website.

Start thinking of final project ideas and dm one line explanation of idea to Yibo, Rishabh, or Nirvan.